Data Engineer

Technical Assignment

Estimated time	3 – 6 hours
Format	Zip file or Git repository

Note: The completed test should be sent at minimum 24 hours before the case review meeting.

Overview

The goal of this assignment is to evaluate how you approach **data ingestion, transformation, and data modeling**. We are interested in how you structure your solution, your reasoning, and the clarity of your code.

You do not need to build a production-ready system, but the solution should reflect good data engineering practices.

Problem

You are given a dataset representing e-commerce transactions. The data comes from multiple sources and needs to be prepared for analytics.

Files provided

File	Description
orders.csv	One row per order
order_items.csv	Line items belonging to each order
customers.csv	Customer master data
products.csv	Product catalog

Data schema

orders.csv

Column	Type	Notes
order_id	string	Primary key
customer_id	string	FK → customers
order_timestamp	timestamp	UTC
order_status	string	completed / cancelled / refunded

order_items.csv

Column	Type	Notes

order_id	string	FK → orders
product_id	string	FK → products
quantity	integer	
price_per_unit	decimal	

customers.csv

Column	Type	Notes
customer_id	string	Primary key
country	string	ISO country name
signup_date	date	

products.csv

Column	Type	Notes
product_id	string	Primary key
category	string	
product_name	string	

Tasks

1

Data Ingestion

Create a process that loads the raw CSV files into a data warehouse or analytical database of your choice (e.g. BigQuery, Snowflake, DuckDB, PostgreSQL, or similar).

• The ingestion process should be repeatable and idempotent where possible.

• Document any assumptions about the target environment.

2

Data Modeling

Design a data model suitable for analytics. This could be a star schema, dimensional model, or another approach you find appropriate.

• Include a brief explanation of your design decisions.

• An ERD or diagram is a nice-to-have but not required.

3 Data Transformations

Create transformations that produce at least the following output datasets:

orders_enriched — One row per order

Column	Description

order_id	
order_timestamp	
customer_id	
country	Joined from customers
order_total_amount	Sum of quantity × price_per_unit
number_of_items	Count of distinct line items

daily_sales — One row per calendar day

Column	Description
date	
total_revenue	
total_orders	Distinct completed orders
average_order_value	

category_sales — One row per product category

Column	Description
category	
total_revenue	
total_quantity_sold	

Note: Consider which order statuses should be included in revenue figures and document your choice.

4 Data Quality

Implement at least **three** data quality checks. Examples:

- Missing or null values in key columns
- Duplicate primary keys
- Invalid or unparseable timestamps
- Negative quantities or prices
- Referential integrity (orders referencing non-existent customers / products)
- Orders with no line items

For each check, briefly describe:

- What it detects
- What action should be taken when it fails (alert, quarantine, fail pipeline, etc.)
- How it would be monitored in a production environment

5 Scalability Discussion

In your README, briefly describe (½–1 page) how your solution would change if the dataset grew from thousands of rows to **billions of rows**. Consider:

- Storage and partitioning strategy
- Incremental vs. full-refresh processing
- Compute and parallelism
- Cost trade-offs

Deliverables

Please submit a zip file or Git repository containing:

- Source code (Python, SQL, dbt, or similar)
- SQL queries or transformation logic
- A README describing how to run the pipeline, design decisions, assumptions, and possible improvements

Optional (nice to have)

- Orchestration example (Airflow, Prefect, Dagster, etc.)
- Unit or integration tests
- Containerisation (Docker / docker-compose)
- Infrastructure as code

Evaluation Criteria

Area	What we look for
Code quality	Structure, readability, consistency
Data modeling	Appropriate schema, clear reasoning
Correctness	Transformation logic matches requirements
Edge case handling	Awareness of dirty data, nulls, duplicates
Documentation	Can another engineer pick this up?
Tradeoffs	Awareness of scalability and production concerns

Tips

- We care more about **clarity and reasoning** than about using a specific tool or database.
- If you make a simplifying assumption, write it down — that's engineering judgment.
- The dataset intentionally contains some data quality issues. How you handle them says a lot.
- A working solution with good documentation beats a half-finished clever one.

Good luck!